

Doc. No.: WG21/P0943R6
Date: 2020-10-23
Reply-to: Hans-J. Boehm
Email: hboehm@google.com
Authors: Hans-J. Boehm
Audience: LWG
Previously reviewed by: SG1, LEWG
Target: C++23

P0943R6: Support C atomics in C++

Abstract

We propose to define what it means to include the C `<stdatomic.h>` header from C++ code. The goal is to enable "shared" headers that use atomics, and can be included from either C or C++ code.

Introduction

As we previously suggested in [P0063R0](#), it would be nice to have a real interoperability story for C atomics and C++ atomics. In the interest of time, and since it appeared to be less essential and more controversial than the rest of that proposal, this suggestion was not pursued further in later revisions of that paper. Nonetheless, it remains a gaping hole in the interoperability story between C and C++. The solution proposed here is based on one that has been used by Android platform code for several years.

The Problem

It is desirable to make C language header files directly usable by C++ code. It is increasingly common for such header files to include declarations that rely on atomics.

For example, a header may wish to declare a function that provides saturating ("clamped") addition on atomic integers. This can easily be implemented in the common subset of C and C++, except that we have no convenient and portable way to include the definition of names such as `atomic_int`, that are defined in `<atomic>` in C++ and in `<stdatomic.h>` in C.

The best we can currently do in portable code is to use an explicit preprocessor test followed by conditional inclusion of either `<atomic>` or `<stdatomic.h>`. In the former case, we then have to add using declarations to inject the required types and functions into the global namespace. Thus we end up with something like

```
#ifdef __cplusplus
#include <atomic>
using std::atomic_int;
using std::memory_order;
using std::memory_order_acquire;
```

```

...
#else /* not __cplusplus */
    #include <stdatomic.h>
#endif /* __cplusplus */
...
int saturated_fetch_add(atomic_int *loc, int value, memory_order order);

```

This approach relies on the currently implicit assumption or wish that the representation of C and C++ atomics are compatible.

Although certainly possible, this is very clumsy compared to the level of interoperability we normally provide; for other language features we commonly provide a C header file that can be included from C++, and provides the necessary declarations. Here we propose to do the same for atomics.

The story here is somewhat confused by the fact that most of the other `.h` compatibility headers currently only appear in the deprecated features section under D.5 C standard library headers [`depr.c.headers`]. A recent paper, [P0619](#) proposes to undeprecate them. We strongly agree with this paper that the current deprecation of these headers does not reflect reality. Many of them are widely used and heavily relied upon. This proposal assumes that at least some of those headers will be preserved.

History

C and C++ atomics were originally designed together. The original design ensured that the non-generic pieces of the C++ atomics library were usable as a C library. This interoperability story became less clear as a result of several later decisions:

- WG14 adopted, in addition to the non-generic parts of the C++ library, a more generic facility, relying on the `_Atomic` type qualifier, and the `_Atomic(T)` type specifier. This happened late enough in the C++11 standardization cycle that there wasn't much of an opportunity for C++ adjustments.
- C++ has never defined what it means to include `stdatomic.h` in a C++ program.

Shortly before finalizing C++11, we briefly discussed making `_Atomic` usable from C++ by defining a macro `_Atomic(T)` as `std::atomic<T>`. An earlier version of this proposal, approved by SG1, did the same. However LEWG did not appear to approve of defining an implementer-namespace name like `_Atomic(T)` in a C++ header. (See below for details.) Hence this version of the proposal uses a different name, on the assumption that WG14 would be willing to introduce a corresponding name for C.

The latter idea was floated on the WG14 reflector, and not immediately shot down. That clearly does not yet amount to approval. But so far we have a weak indication that it may be preferred over the alternative of simply dropping anything like `_Atomic` from the C and C++ common subset.

Proposal

We propose to add a header `stdatomic.h` to the C++ standard. This would mirror the identically named C header, and provide comparable functionality. However, instead of defining it in terms of the C header, we propose that it have the following effects:

1. Include the `<atomic>` header.
2. Define a macro such as `atomic_generic_type(T) as std::atomic<T>`. This assumes that C would correspondingly define `atomic_generic_type(T) as _Atomic(T)`.
3. Promote all `atomic_...` and `memory_order...` identifiers introduced by the `<atomic>` header into the global name space.

Although this provides functionality very similar to the C header, it is unlikely that it will be exactly the same. As usual, it is the responsibility of the author of a C and C++ shared header to ensure that the code is correct in both languages.

This proposal makes no effort to support an equivalent of the C `_Atomic` type qualifier, as opposed to the type *specifier* (which is spelled with parentheses). This is intentional; accommodating the qualifier would be a major language change, particularly since it is the only type qualifier that can affect alignment.

We believe this is the minimum required to conveniently use atomics in shared headers.

Implementation compatibility

Aside from the renaming of `_Atomic`, this proposal reflects current practice in Android platform code since the Android Lollipop release in 2014. (See <https://android.googlesource.com/platform/bionic/+lollipop-release/libc/include/stdatomic.h>. That uses a single include file shared between C and C++, which may not be the optimal implementation strategy.)

In an ideal world, this would be easy to support in various compiler environments. The main requirement is that C and C++ atomics should be implemented identically and use the same ABI conventions. There is no fundamental reason to do anything differently. Implementing ordering constraints in incompatible ways is already greatly undesirable, in that mixed language programs with `memory_order` annotations would no longer be sequentially consistent. Using incompatible locking schemes for large atomics would generally double the amount of space required for the lock table, and complicate ABI conventions, with no benefit.

Our preliminary experiments with gcc-4.9 found no differences on x86-64, though it did find some seemingly gratuitous alignment differences on ARMv7. Unfortunately later information from Jonathan Wakely and David Goldblatt points out that gcc in fact relies on two separate mechanisms for determining atomics alignment in C and C++, and these differ in some corner cases, since alignment decisions are made by the compiler in one case and the library the other. Although this is nontrivial to change, and requires C++ ABI changes in those corner cases, those changes appear desirable, even independent of this proposal. Some applications already assume compatible C++ and C ABIs for atomics, and those will currently break. And the current approach appears to preclude a consistent ABI between e.g. gcc and clang. There appears to be general agreement that this should be fixed in any case. More discussion can be found at https://gcc.gnu.org/bugzilla/show_bug.cgi?id=71660.

Thus the implementation effort for meaningful conformance to this proposal will vary between adding a simple header file on one hand, and appreciable changes to the existing atomics alignment logic in either C or C++ on the other. The implementation effort for minimal conformance consists of only the header file, since we can't normatively say anything about C and C++ consistency.

Should there be a `<cstdatomic>`?

We do not believe there is a strong need to introduce a `<cstdatomic>` that only introduces names into the `std::` namespace. The justification for `<stdatomic.h>` is entirely to support headers shared between C and C++, not to import a C facility into C++. A shared header will not be able to take advantage of names introduced into the `std::` namespace.

There is an argument that `<cstdatomic>` should be provided anyway for uniformity. It's not clear that this outweighs the cost of introducing a header that we expect to get essentially no use. There did not appear to be sentiment in SG1 for adding `<cstdatomic>`.

Wording

I propose to add a new section to the standard, placed at the editor's discretion. The general intent is to place it in the same section as other C compatibility headers that we expect to keep around indefinitely. This facility is not intended for future removal. Unfortunately, many other C headers are currently listed as deprecated, in spite of what appears to be a consensus that they are not destined for future removal either. Note that [P0619](#) discusses this issue in more detail.

If we keep the current organization, LWG seemed to favor placing this in the atomics section over Appendix D. LEWG was split between those. A new section for all such compatibility features seems like another, perhaps better, option.

Add a new section:

XX.Y.Z C Compatibility for atomics [atomics.compat]

The header `<stdatomic.h>` provides the following definitions:

```
template<class T>  
using std-atomic = std::atomic<T>; // exposition only  
#define Atomic(T) std-atomic<T>
```

```
#define ATOMIC_BOOL_LOCK_FREE see below  
#define ATOMIC_CHAR_LOCK_FREE see below  
#define ATOMIC_CHAR16_T_LOCK_FREE see below  
#define ATOMIC_CHAR32_T_LOCK_FREE see below  
#define ATOMIC_WCHAR_T_LOCK_FREE see below  
#define ATOMIC_SHORT_LOCK_FREE see below  
#define ATOMIC_INT_LOCK_FREE see below  
#define ATOMIC_LONG_LOCK_FREE see below  
#define ATOMIC_LLONG_LOCK_FREE see below  
#define ATOMIC_POINTER_LOCK_FREE see below
```

```
using std::memory_order; // see below  
using std::memory_order_relaxed; // see below  
using std::memory_order_consume; // see below
```

using std::memory_order_acquire; // see below
using std::memory_order_release; // see below
using std::memory_order_acq_rel; // see below
using std::memory_order_seq_cst; // see below

using std::atomic_flag; // see below

using std::atomic_bool; // see below
using std::atomic_char; // see below
using std::atomic_schar; // see below
using std::atomic_uchar; // see below
using std::atomic_short; // see below
using std::atomic_ushort; // see below
using std::atomic_int; // see below
using std::atomic_uint; // see below
using std::atomic_long; // see below
using std::atomic_ulong; // see below
using std::atomic_llong; // see below
using std::atomic_char8_t; // see below
using std::atomic_char16_t; // see below
using std::atomic_char32_t; // see below
using std::atomic_wchar_t; // see below
using std::atomic_int8_t; // see below
using std::atomic_uint8_t; // see below
using std::atomic_int16_t; // see below
using std::atomic_uint16_t; // see below
using std::atomic_int32_t; // see below
using std::atomic_uint32_t; // see below
using std::atomic_int64_t; // see below
using std::atomic_uint64_t; // see below
using std::atomic_int_least8_t; // see below
using std::atomic_uint_least8_t; // see below
using std::atomic_int_least16_t; // see below
using std::atomic_uint_least16_t; // see below
using std::atomic_int_least32_t; // see below
using std::atomic_uint_least32_t; // see below
using std::atomic_int_least64_t; // see below
using std::atomic_uint_least64_t; // see below
using std::atomic_int_fast8_t; // see below
using std::atomic_uint_fast8_t; // see below
using std::atomic_int_fast16_t; // see below
using std::atomic_uint_fast16_t; // see below
using std::atomic_int_fast32_t; // see below
using std::atomic_uint_fast32_t; // see below

using std::atomic int fast64 t; // see below
using std::atomic uint fast64 t; // see below
using std::atomic intptr t; // see below
using std::atomic uintptr t; // see below
using std::atomic size t; // see below
using std::atomic ptrdiff t; // see below
using std::atomic intmax t; // see below
using std::atomic uintmax t; // see below

using std::atomic is lock free; // see below
using std::atomic load; // see below
using std::atomic load explicit; // see below
using std::atomic store; // see below
using std::atomic store explicit; // see below
using std::atomic exchange; // see below
using std::atomic exchange explicit; // see below
using std::atomic compare exchange strong; // see below
using std::atomic compare exchange strong explicit; // see below
using std::atomic compare exchange weak; // see below
using std::atomic compare exchange weak explicit; // see below
using std::atomic fetch add; // see below
using std::atomic fetch add explicit; // see below
using std::atomic fetch sub; // see below
using std::atomic fetch sub explicit; // see below
using std::atomic fetch or; // see below
using std::atomic fetch or explicit; // see below
using std::atomic fetch and; // see below
using std::atomic fetch and explicit; // see below
using std::atomic flag test and set; // see below
using std::atomic flag test and set explicit; // see below
using std::atomic flag clear; // see below
using std::atomic flag clear explicit; // see below

using std::atomic thread fence; // see below
using std::atomic signal fence; // see below

Each *using-declaration* for *A* in the synopsis above makes available the same entity as `std::A` declared in `<atomic>`. Each macro listed above other than `Atomic(T)` is defined as in `<atomic>`. It is unspecified whether `<stdatomic.h>` makes available any declarations in namespace `std`.

Each of the *using-declarations* for `intN t`, `uintN t`, `intptr t`, and `uintptr t` listed above is defined if and only if the implementation defines the corresponding typedef name in `[atomics.syn]`.

Neither the `Atomic` macro, nor any of the non-macro global namespace declarations are provided by any C++ standard library header other than `<stdatomic.h>`.

Recommended Practice: Implementations should ensure that C and C++ representations of atomic objects are compatible, so that the same object can be accessed as both an `Atomic(T)` from C code and `atomic<T>` from C++ code. The representations should be the same, and the mechanisms used to ensure atomicity and memory ordering should be compatible.

Add a feature test macro in 17.3.2 [version.syn]:

```
#define __cpp_lib_stdatomic_h 202XYYL // also in <stdatomic.h>
```

The above was originally based on the part of the Android `<stdatomic.h>` header that is used when compiling platform C++ code.

Discussion so far and open issues

R0, with changes that were incorporated into R1, was approved by SG1 in Jacksonville, 6/17/4/10/0. R1 was not discussed in Rapperswil, since LEWG was bandwidth-constrained.

R1 was briefly discussed by LEWG in San Diego. LEWG voted mildly against putting "`_Atomic(T)` ... in the set of tokens that are cross-language compatible" and, in a separate poll, mildly against forwarding to LWG. There was a consensus in favor of "Revise proposal along the lines of `effects equivalent to (the goals of this paper)`".

The concerns I heard, both during the meeting, and during a later discussion with Titus Winters were:

1. LEWG doesn't like to encourage programming with identifiers starting with `<underscore><capital>`, i.e. identifiers in the system namespace.
2. Clang already assigns a meaning to `_Atomic` in C++.
3. `_Atomic(T)` and `atomic_int` in C++ would yield objects that have member functions, which differs from C.

These unfortunately raised some additional questions, which we did not have a chance to pursue until the Cologne meeting:

1. An obvious solution to all objections except (3) would be to drop the definition of `_Atomic`. (Since the identifier is in the system namespace, implementations could choose to keep it anyway.) Would this resolve LEWG concerns? Would it reduce SG1 consensus? There is weak evidence that this is unpopular in WG14.
2. The other possible resolution to the first point would be to introduce a new alias for `_Atomic` in C, as proposed in R2. Is this satisfactory for SG1? LEWG? WG14?
3. It would be nice to get clarification of the concern about `_Atomic` in clang. It is true that clang already assigns a meaning to `_Atomic`. But it is not clear that this breaks anything, since we would effectively be replacing that with a presumed compatible implementation if `stdatomic.h` is included in C++ code.
4. Similarly, I didn't understand the last concern, and the extent to which it's important to LEWG. `atomic_int` in C++ already contains member functions in C++, which does differ from C. But this seems entirely unavoidable, and was part of the original C/C++ atomics design, which was intended to support the kind of compatibility we're looking for here. Furthermore, we clearly want the standalone C++ atomics functions to be applicable to C-compatible atomics. That requires that C atomics be convertible to

`atomic<T>`, which makes it hard to see how `atomic_int`, or `_Atomic(int)`, if we keep it, could not have member functions in C++.

During follow-up discussion in Cologne, using R3, which replaced the `_Atomic` define with another name, and based on the preceding points, SG1 expressed a preference to restore the macro name to `_Atomic`. LEWG agreed, and agreed to forward this to LWG. An LEWG vote on whether to put this into Appendix D was entirely inconclusive.

History

Changed in R1

Changes made to reflect SG1 discussion Jacksonville. SG1 voted 6/17/4/1/0 to advance to to LEWG, but suggested a few changes:

- Removed false claim in discussion that this is always easy to implement.
- Promote only types to global name space, and let ADL take care of the rest. Android platform code promotes all of them, but this now seems suboptimal.
- Explicitly list all `using` declarations required for the namespace promotion.

Note that the SG1 discussion was slightly incorrect in that it overlooked the recent change of `memory_order` to an `enum class`. But in retrospect it's not at all clear that this affects our proposal. It simply adds one more easily avoidable incompatibility between C and C++ usages of the header.

Added in R2

Revised the proposal to define `atomic_generic_type` (placeholder name) instead of `_Atomic`. Added the last section about the LEWG discussion in San Diego.

Changed in R3

Corrected angle brackets to parentheses in the wording of the note. (Pointed out on the reflector by Jonathan Wakely.) Added abstract.

Changed in R4

Based on SG1 and LEWG direction in Cologne, changed macro name back to `_Atomic`. Added suggestion for new appendix. Added `atomic_char8_t`, and improved the wording, both based on Daniel Krügler's suggestions.

Changed in R5

Clarified / corrected the use of `stdatomic.h` in Android C++ code. I was previously mistaken about the use of this header outside the platform code. It is used by code in the platform itself, but is not currently exported through the NDK to client code. Note that the platform itself still includes a large amount of C++ code.

Changed in R6

Changes in response to LWG comments:

- Changed recommendation to add this to the main body of the standard.
- Fixed the compatibility note to again refer to `_Atomic()`.
- Added feature test macro.
- Changed note to recommended practice clause.
- Rewrote much of the wording to not require `<stdatomic.h>` to include `<atomic>`, relying heavily on advice from many LWG participants.
- Added prohibition against including `<stdatomic.h>` from other C++ headers; C already has such a prohibition.